Mahidol University
Wisdom of the Land

Mahidol University

Lecture 05: Repetition - I

Lecturers:
Aj. Jidapa Kraisangka
Aj. Akara Supratak
Aj. Suradej Intagprn

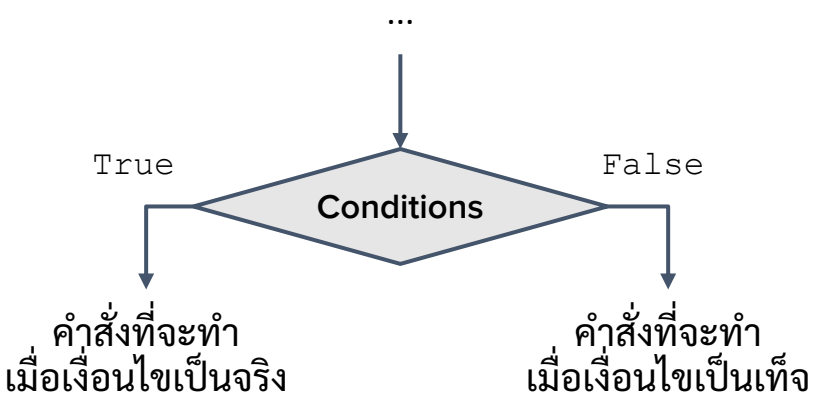
Topics

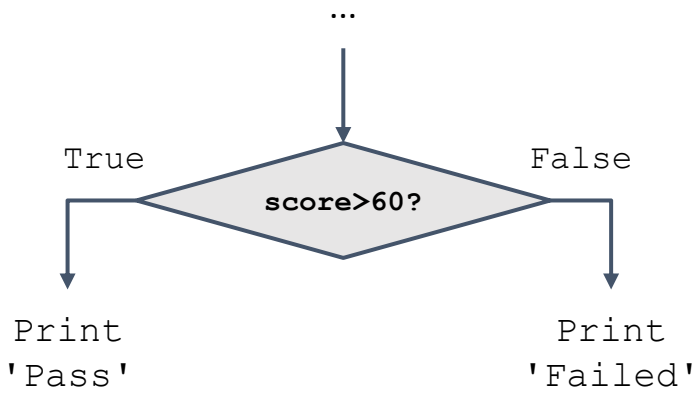
- **RECAP: relational and logical expressions, `if-else` statement, and `elif` statement**
- **Repetition:**
  - `while` **statement**
  - `for` **statement**
  - **Loop control statements:** `continue` **and** `break`

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

RECAP: Selection

- **Selection:** รูปแบบที่การเลือกทำชุดคำสั่งบางชุด โดยพิจารณาจากค่าที่เงื่อนไข (Conditions) เหล่านั้นเป็นจริง(`True`) หรือเท็จ (`False`)





RECAP: Relational Expression

Relational expression คือ expressions ที่ใช้ตัวดำเนินการเชิงเปรียบเทียบ (Comparison operators) โดยจะเปรียบเทียบค่าสองค่าและประมวลผลลัพธ์เป็นค่า Boolean

Table 2-1: Comparison Operators

Operator	Meaning
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

เครื่องหมาย == ใช้เปรียบเทียบว่าค่าสองค่าเท่ากันหรือไม่ ซึ่งไม่ใช่ Assignment operator (=) ที่นำค่ามาใส่ตัวแปร

ใช้เปรียบเทียบเฉพาะ `int` และ `float` เท่านั้น

Table from https://automatetheboringstuff.com/2e/chapter2/

Logical Expression

Logical expression คือ expressions ที่ใช้ตัวดำเนินการด้านตรรกศาสตร์ (Boolean operators ได้แก่ **and**, **or** และ **not**) โดยจะเปรียบเทียบค่า Boolean และประมวลผลลัพธ์เป็นค่า **Boolean (`True` or `False`)**

Table 2-2: The and Operator's Truth Table

Expression	Evaluates to . . .
True and True	True
True and False	False
False and True	False
False and False	False

Table 2-3: The or Operator's Truth Table

Expression	Evaluates to . . .
True or True	True
True or False	True
False or True	True
False or False	False

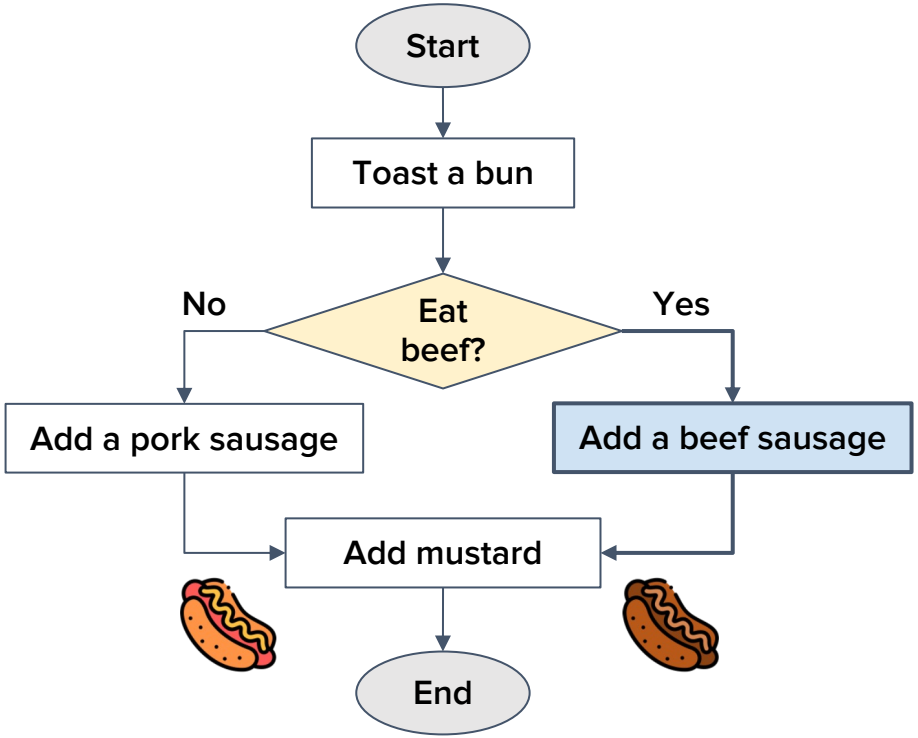
Table 2-4: The not Operator's Truth Table

Expression	Evaluates to . . .
not True	False
not False	True

Tables from https://automatetheboringstuff.com/2e/chapter2/

RECAP: Conditions

- Relational Expression และ Logical Expression เป็นพื้นฐานสำหรับการเขียนเงื่อนไข (Conditions)
- Conditions จะถูกประมวลผลเป็นค่า Boolean เสมอ
- ผลของ Conditions (ได้แก่ True, False) จะเป็นตัวกำหนดชุดคำสั่งที่จะกระทำ



Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

RECAP: `if-else` statement

- **`if`** เป็นคำสั่งที่ใช้ควบคุมการทำงานของโปรแกรมให้ทำงานตามเงื่อนไขที่กำหนดเมื่อเงื่อนไขนั้น ๆ เป็นจริง
- **`else`** เป็นคำสั่งที่ใช้ต่อเนื่องจากคำสั่ง **if** เพื่อควบคุมการทำงานของโปรแกรมให้ทำงานตามเงื่อนไขที่กำหนดเมื่อเงื่อนไขนั้น ๆ เป็นเท็จ

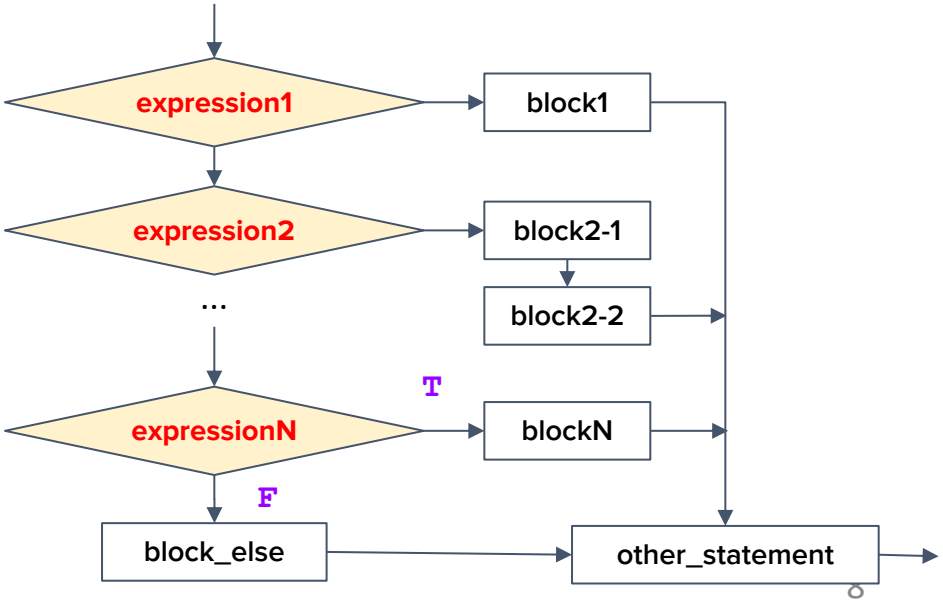
```
normal_statementX

if expression:
        statement1
        ...
        statementN

normal_statementY
```

```
normal_statementX

if expression:
        statement_when_T
else :
    statement_when_F

normal_statementY
```

RECAP: elif statement

- **elif** (ย่อจาก else if) เป็นคำสั่งที่ใช้ต่อเนื่องจากคำสั่ง **if** เพื่อควบคุมการทำงานของโปรแกรมให้ทำงานตาม<u>หลาย ๆ เงื่อนไขอย่างต่อเนื่อง</u>
- ใช้ร่วมกับ **if** และ **else**

```
if expression1:
        statement_block1
elif expression2:
        statement_block2-1
    statement_block2-2
...
elif expressionN:
        statement_blockN
else:
    statement_block_else
other_statement
```



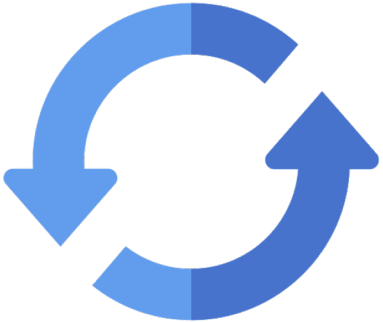
Flow Controls

- **Flow controls (in programming)** เป็นลักษณะการเขียนโปรแกรม ที่มีลำดับการประมวลผลแตกต่างกันไป

- รูปแบบของ Flow controls ได้แก่
  - **Sequential:** รูปแบบปกติที่เป็นลำดับขั้นตอนเรียงกัน
  - **Selection:** รูปแบบที่การเลือกทำชุดคำสั่งบางชุดตามเงื่อนไข
  - **Repetition:** รูปแบบปกติที่ทำซ้ำในบางชุดคำสั่ง
  - **Invocation:** รูปแบบการเรียกใช้ชุดคำสั่งอื่น ๆ หรือฟังก์ชัน

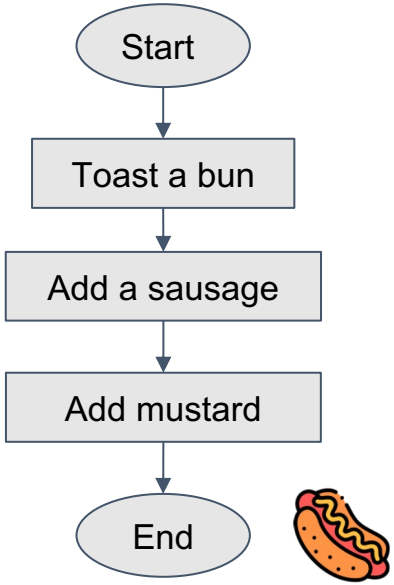
Repetition

- **Repetition** หรือการเขียนโปรแกรมแบบวนซ้ำ
- เป็นการนำชุดคำสั่งมาทำงานซ้ำหลายๆ รอบ
- จะทำงานกี่รอบขึ้นอยู่กับเงื่อนไขที่ผู้เขียนโปรแกรมกำหนดไว้
- ชุดคำสั่งที่มีการวนใช้ซ้ำๆ เรียกว่า **loop**

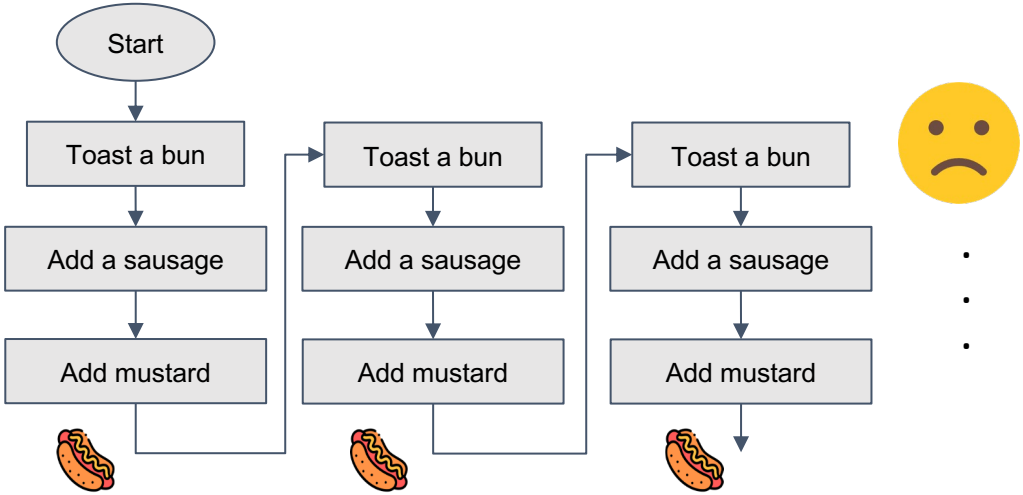


Why Loop Is Needed?

ขั้นตอนการทำ hotdog 1 อัน



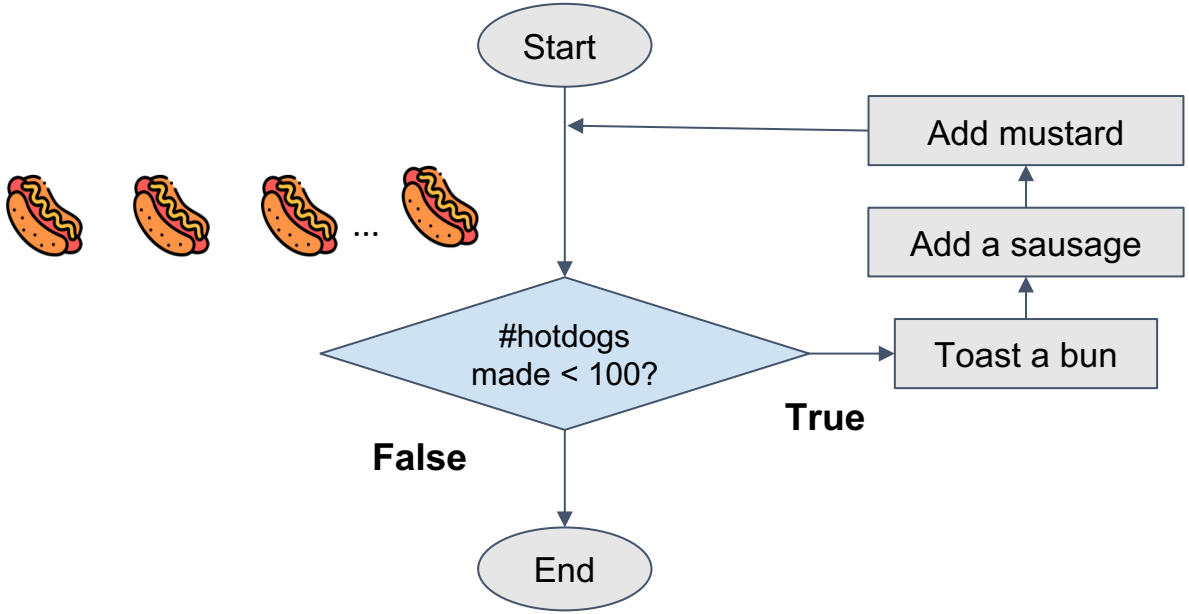
ถ้าต้องการทำ hotdog 100 หรือ 1000 อัน
เราควรจะต้องเขียนชุดคำสั่งนี้ 100 หรือ 1000 ครั้ง?



Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

Loop Example

การใช้ loop ช่วยให้การเขียน code มีความ flexible และสั้นลง



Loop จะวนไปเรื่อยๆ ถ้า condition ที่กำหนดเป็นจริง

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

Repetition Statements

คำสั่งที่ใช้ในการวน loop

- `while` statement
- `for` statement

while Statement

- ควบคุมโปรแกรมให้ทำงานบางอย่างซ้ำๆ ในขณะที่เงื่อนไข (expression) ของ loop นั้นยังคงเป็นจริง (ไม่เท่ากับ 0) อยู่
- มักถูกใช้เมื่อจำนวนรอบในการวน loop ไม่ทราบล่วงหน้า

```
while expression:
        statement(s)
```

ขั้นตอนการทำงานของ `while` loop:

1. Test the expression
2. If the expression is TRUE
   a. Execute the statement(s)
   b. Go to step 1

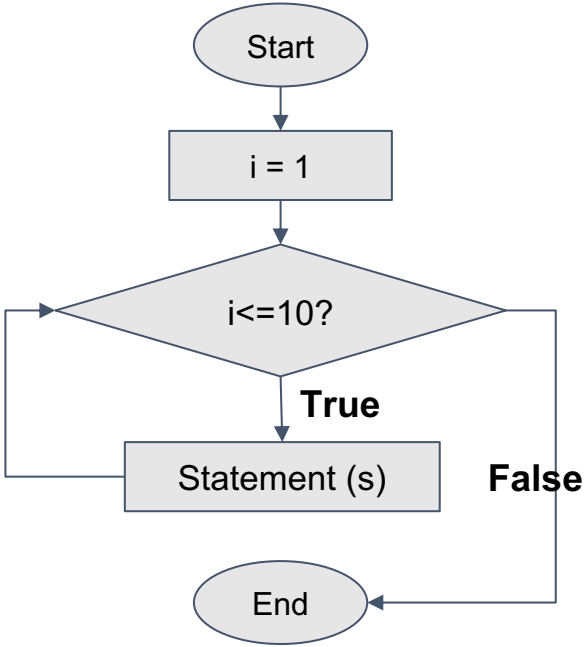
   Else exit the `while` loop

Basic `while` Structure

1. **Conditions**
   เงื่อนไขที่ใช้การวน loop ถ้าเป็นจริง loop จะทำงานซ้ำ โดยใช้ (Relational/Logical) Expression
2. **Initialization:**
   การกำหนดค่าเริ่มต้นของตัวแปรที่จะเป็นเงื่อนไขในการ loop
3. **Alteration:**
   - การเปลี่ยนแปลงค่าของตัวแปรที่ใช้เป็นเงื่อนไขในการ loop
   - ผลลัพธ์ของ expression จะถูกเปลี่ยนเป็น **False** เพื่อหยุดการทำงานของ loop

while Loop: Example

Print เลข 1 ถึง 10



Initialization

Conditions จะต้องเป็น **True** เพื่อให้ loop ทำงานต่อไป

```
i = 1
while i<=10:
        print(i)
        i=i+1
```

Alteration

Output:
1
2
3
4
5
...
10

การทำงานของ Loop


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

[redacted]

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
2
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2
2	2	True	3

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
2
3
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2
2	2	True	3
3	3	True	4

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
2
3
4
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2
2	2	True	3
3	3	True	4
4	4	True	5

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
2
3
4
5
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2
2	2	True	3
3	3	True	4
4	4	True	5
5	5	True	6

การทำงานของ Loop (ต่อ)


```
1. i=1
2. while i<=5:
3.     print(i)
4.     i=i+1
5. print('End')
```

Output:

```
1
2
3
4
5
End
```

รอบที่	ค่า i ที่เงื่อนไข (Line#2)	i<=5 (Line#2)	ค่า i ที่ Line#4
1	1	True	2
2	2	True	3
3	3	True	4
4	4	True	5
5	5	True	6
6	6	False	-

เมื่อมีการตรวจสอบเงื่อนในรอบที่ 6 เงื่อนไข `i<=5` มีค่าเป็น `False` ทำให้การ while loop หยุดการทำงาน แล้วทำคำสั่งต่อไปนอก while block ใน Line#6

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

Exercise: Understanding Loops

จงระบุจำนวนรอบ (Loop) และผลลัพธ์ในแต่ละรอบที่โปรแกรมทำงาน จากส่วนของโปรแกรมต่อไปนี้

ส่วนของโปรแกรม	จำนวนรอบ (Loop)	ผลลัพธ์ในแต่ละรอบ

`i=1`
`while i<=3:`
`  print(1)`
`  i=i+1`
	3	Loop#1: 1 `(i=1)`
		Loop#2: 1 `(i=2)`
		Loop#3: 1 `(i=3)`

`i=1`
`while i<=10:`
`  print(i)`
`  i=i+1`		

`i=0`
`while i<10:`
`  print(i)`
`  i=i+1`		

Exercise: Understanding Loops

จงระบุจำนวนรอบ (Loop) และผลลัพธ์ในแต่ละรอบที่โปรแกรมทำงาน จากส่วนของโปรแกรมต่อไปนี้

ส่วนของโปรแกรม	จำนวนรอบ (Loop)	ผลลัพธ์ในแต่ละรอบ

`i=0`
`x=0`
`while i<20:`
`  print(x, end=' ')`
`  x=x+1`
`  i=i+5`		

`i=8`
`x=5`
`while i>2:`
`  print(x, end=' ')`
`  x=x+1`
`  i=i-2`		

Exercise: Condition for Loops

จงเขียนเงื่อนไขใน `while` ที่ทำให้โปรแกรมแสดงผลลัพธ์ตามต้องการดังต่อไปนี้

ส่วนของโปรแกรม	ผลลัพธ์ที่ต้องการ	เงื่อนไข

`i=1`
`while [redacted]:`
`  print(1, end=' ')`
`  i=i+1`
	1 1 1 1 1	i<=5

`x=-2`
`while [redacted]:`
`  print(x, end=' ')`
`  x=x+1`
	-2 -1 0 1 2 3 4	

`x=0`
`while [redacted]:`
`  if x%10==0:`
`  print(x, end=' ')`
`  x=x+1`
	0 10 20 30 40 50	

while Loop: Exercise

จงเขียน while ในการแสดงผลลัพธ์ตัวเลขต่อไปนี้

- 100 99 98 … 1
- 9 7 5 3 1
- 1 2 4 8 16 32 64
- 1 2 4 7 11 16 22 29 37
- 10 x 9 y 8 x 7 y 6 x 5 y 4 x 3 y 2 x 1 y

while Loop: Exercise

Q1: 100 99 98 … 1

Q2: 1 3 5 7 9

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

while Loop: Exercise

Q3: 1 2 4 8 16 32 64

Q4: 1 2 4 7 11 16 22 29 37

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

while Loop: Exercise

Q5: 10 x 9 y 8 x 7 y 6 x 5 y 4 x 3 y 2 x 1 y

Infinite Loop

- การใช้ **`while` statement** ควรใช้ด้วยความระมัดระวัง เนื่องจากสามารถทำให้เกิด loop ที่ไม่สิ้นสุดได้
- เกิดจากการใช้ expression ที่ผิด หรือการลืมเปลี่ยนแปลงค่าตัวแปรที่ใช้เป็นเงื่อนไขในการ loop → condition ไม่ถูกเปลี่ยนให้เป็น False
- ในกรณีที่โปรแกรมวน loop ไม่หยุด กด **ctrl +c**

```
i = 1
while i>0:
        print(i)
        i +=1
```

Incorrect expression

This loop will never end

`for` statement

for Statement

- เป็นคำสั่งวนซ้ำที่ใช้ควบคุมการทำงานซ้ำๆ มักใช้สำหรับการวนอ่านค่าใน iterable object เช่น list, string, tuple
- มักใช้เมื่อทราบจำนวนรอบในการวน loop แน่นอน

```
for var in <sequence>:
    statement(s)
```

ขั้นตอนการทำงาน for loop:

1. Item แรกใน sequence ถูก assign เก็บใน **var**
2. ชุดคำสั่งใน for loop ถูก execute
3. If item ใน sequence ยังมีอยู่
   a. item ถัดมาใน sequence จะถูก assign ให้ **var**
   b. กลับไป step 2

   Else ออกจาก for loop

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

range() Function

- `range()` เป็น built-in function ที่มักใช้ร่วมกับ for loop
- ใช้ในการสร้าง sequence ของตัวเลข ภายใน range ที่กำหนด
- Default: เริ่มต้นที่ 0, เพิ่มขึ้นทีละ 1, และจบที่ตัวเลขสุดท้ายที่กำหนด

Syntax:

```
range(stop)
range(start, stop)
range(start, stop, step)
```

มี parameter 3 ตัว เป็น type integer ได้แก่

- `start:` ตัวเลขเริ่มต้น
- `stop:` ตัวเลขสุดท้าย (ไม่รวม)
- `step:` ค่าที่เปลี่ยนแปลง

ไม่ support float numbers

for/range(): Example

ไม่รวมตัวสุดท้าย

```
for x in range(10):
    print(x, end=' ')
```

Output:

```
0 1 2 3 4 5 6 7 8 9
```

```
for x in range(5, 12):
    print(x, end=' ')
```

```
5 6 7 8 9 10 11
```

for/range(): Example

increment/เพิ่ม

```
for x in range(1, 9, 1):
    print(x, end=' ')
```

```
1 2 3 4 5 6 7 8
```

```
for x in range(1, 15, 3):
    print(x, end=' ')
```

```
1 4 7 10 13
```

```
for x in range(8, 0, -1):
    print(x, end=' ')
```

```
8 7 6 5 4 3 2 1
```

decrement/ลด

for/range(): Exercise

หา output ของ code ต่อไปนี้:

Output

```
for x in range(7):
        print(x, end=' ')
```

```
for x in range(1, 11):
        print(x, end=' ')
```

```
for x in range(-5, 10, 2):
        print(x, end=' ')
```

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

for/range(): Exercise

หา output ของ code ต่อไปนี้:

Output

```
for x in range(-5):
        print(x, end=' ')
```

```
for x in range(5, -4, -1):
        print(x, end=' ')
```

Exercise: Understanding Loops

จงระบุจำนวนรอบ (Loop) และผลลัพธ์ในแต่ละรอบที่โปรแกรมทำงาน จากส่วนของโปรแกรมต่อไปนี้

ส่วนของโปรแกรม	จำนวนรอบ (Loop)	ผลลัพธ์ในแต่ละรอบ
`for x in range(3):` `  print(x, end=' ')`	3	Loop#1: 1 (x=1) Loop#2: 1 (x=2) Loop#3: 1 (x=3)

`q=1;`
`for x in range(0, 6):`
`  print(q, end=' ')`
`  q=q*2;`		

`for x in range(10):`
`  if x%2==1:`
`  print(x, end=' ')`
`  else:`
`  print('-', end=' ')`		

for/range(): Exercise

เขียน **for** loop เพื่อ print output ดังต่อไปนี้

- `100 99 98 … 1`
- `9 7 5 3 1`
- `1 2 4 8 16 32`
- `20 16 12 8 4 0 -4 -8 -12 -16 -20`
- `a b a b a b a b`

for/range(): Exercise

Q1: 100 99 98 … 1

Q2: 9 7 5 3 1

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

for/range(): Exercise

Q3: 1 2 4 8 16 32

Q4: 20 16 12 8 0 4 -4 -8 -12 -16 -20

Contact: [jidapa.kra, akara.sup, suradej.int]@mahidol.edu

for/range(): Exercise

Q5: a b a b a b a b

ตัวอย่างการใช้ Loop

Example: Try while loop

เขียนโปรแกรมรับ input ตัวเลข 10 จำนวน และ print ผลรวมของตัวเลข

Output:

```
Enter a number  : 10
10
Enter a number  : 5
15
Enter a number  : 3
18
Enter a number  : 12
30
...
```

Example: Try for loop

เขียนโปรแกรมรับ input ตัวเลข 10 จำนวน และ print ผลรวมของตัวเลข

Output:

```
Enter a number  : 10
10
Enter a number  : 5
15
Enter a number  : 3
18
Enter a number  : 12
30
...
```

Exercise: Sum until negative

ไม่ทราบจำนวนการวนซ้ำล่วงหน้า

จงเขียนโปรแกรมเพื่อรับ input ตัวเลข จนกว่าตัวเลขที่รับจะเป็นจำนวนลบ จากนั้นแสดงผลรวมของ input ที่เป็นจำนวนบวกทั้งหมด

Output:

```
Enter a number  : 10
10.0
Enter a number  : 5
15.0
Enter a number  : 3
18.0
Enter a number  : 0
18.0
Enter a number  : -1
```

Output:

```
Enter a number  : -5
```

Output:

```
Enter a number  : 5
5.0
Enter a number  : 5
10.0
Enter a number  : -3
```

Exercise: Sum until negative

Output:

```
Enter a number  : 10
10.0
Enter a number  : 5
15.0
Enter a number  : 3
18.0
Enter a number  : 0
18.0
Enter a number  : -1
```

while vs for

- **`for`**: มักใช้เมื่อทราบจำนวนรอบในการ loop อย่างแน่นอน
  - ใช้ร่วมกับ **function** `range()` และ **iterable objects** เช่น **list, string, tuple**
- **`while`:** มักใช้เมื่อจำนวนรอบในการ loop ไม่ทราบล่วงหน้า
  - เช่น โปรแกรมที่จำนวน loop ขึ้นกับ user input
  - แต่สามารถใช้กับกรณีทราบจำนวนรอบในการ loop ล่วงหน้าแล้วเช่นกัน ขึ้นอยู่กับ style การ code ของ programmer

Thanks to: https://keep-calm.net/keep-calm-and-code-on.html

Happy Coding :)
More Loop next week